

C++ EXCEPTION HANDLING

http://www.tutorialspoint.com/cplusplus/cpp_exceptions_handling.htm

Copyright © tutorialspoint.com

```
try
{
    // protected code
} catch( ExceptionName e1 )
{
    // catch block
} catch( ExceptionName e2 )
{
    // catch block
} catch( ExceptionName eN )
{
    // catch block
}
```

```
double division(int a, int b)
{
    if( b == 0 )
    {
        throw "Division by zero condition!";
    }
    return (a/b);
}
```



```
try
{
    // protected code
} catch( ExceptionName e )
{
    // code to handle ExceptionName exception
}
```

Above code will catch an exception of **ExceptionName** type. If you want to specify that a catch block should handle any type of exception that is thrown in a try block, you must put an ellipsis, ..., between the parentheses enclosing the exception declaration as follows:

```
try
{
    // protected code
} catch(...)
{
    // code to handle any exception
}
```

The following is an example, which throws a division by zero exception and we catch it in catch block.

```
#include <iostream>
using namespace std;

double division(int a, int b)
{
    if( b == 0 )
    {
        throw "Division by zero condition!";
    }
    return (a/b);
}

int main ()
{
    int x = 50;
    int y = 0;
    double z = 0;

    try {
        z= division(x, y);
        cout<< z << endl;
    } catch (const char* msg) {
        cerr<< msg << endl;
    }

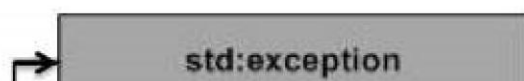
    return 0;
}
```

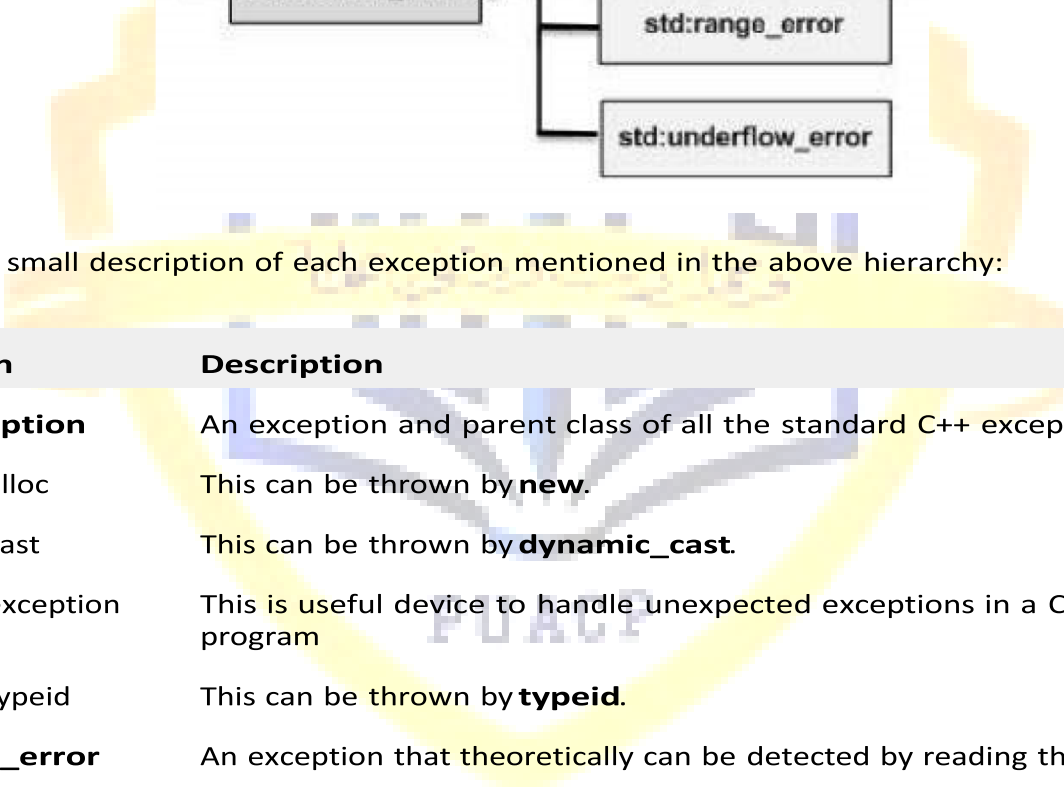
Because we are raising an exception of type **const char***, so while catching this exception, we have to use **const char*** in catch block. If we compile and run above code, this would produce the following result:

```
Division by zero condition!
```

C++ Standard Exceptions:

C++ provides a list of standard exceptions defined in **<exception>** which we can use in our programs. These are arranged in a parent-child class hierarchy shown below:





small description of each exception mentioned in the above hierarchy:

Exception	Description
std::exception	An exception and parent class of all the standard C++ exceptions.
std::bad_alloc	This can be thrown by new .
std::bad_cast	This can be thrown by dynamic_cast .
std::bad_exception	This is useful device to handle unexpected exceptions in a C++ program
std::bad_typeid	This can be thrown by typeid .
std::logic_error	An exception that theoretically can be detected by reading the code.
std::domain_error	This is an exception thrown when a mathematically invalid domain is used
std::invalid_argument	This is thrown due to invalid arguments.
std::length_error	This is thrown when a too big std::string is created
std::out_of_range	This can be thrown by the at method from for example a std::vector and std::bitset<>::operator[] .
std::runtime_error	An exception that theoretically can not be detected by reading the code.
std::overflow_error	This is thrown if a mathematical overflow occurs.
std::range_error	This is occurred when you try to store a value which is out of range.
std::underflow_error	This is thrown if a mathematical underflow occurs.

Define New Exceptions:

You can define your own exceptions by inheriting and overriding **exception** class functionality. Following is the example, which shows how you can use `std::exception` class to implement your own exception in standard way:

```
#include <iostream>
#include <exception>
using namespace std;

struct MyException : public exception
{
    const char * what () const throw ()
    {
        return "C++ Exception";
    }
};

int main()
{
    try
    {
        throw MyException();
    }
    catch(MyException& e)
    {
        std::cout << "MyException caught"<< std::endl;
        std::cout << e.what() << std::endl;
    }
    catch(std::exception& e)
    {
        //Other errors
    }
}
```

This would produce the following result:

```
MyException caught
C++ Exception
```

Here, **what** is a public method provided by exception class and it has been overridden by all the derived classes. This method returns the cause of an exception.